

PyTorch 基础及应用

💡 这节课会带给你

- ☒ 机器学习基础：核心概念、反向传播与自动求导
- ☒ PyTorch从线性回归到多层感知机的训练与优化
- ☒ PyTorch计算图生成、优化与高级 API

🎓 这节课怎么学

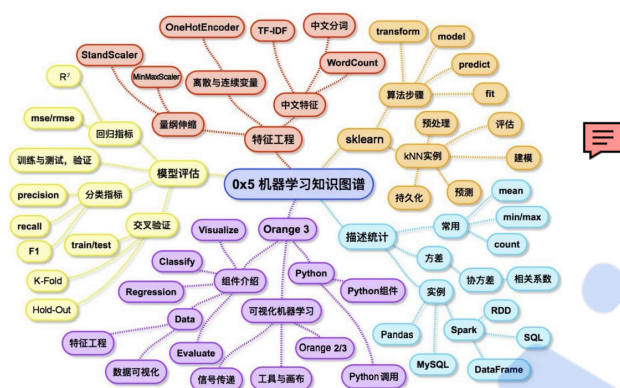
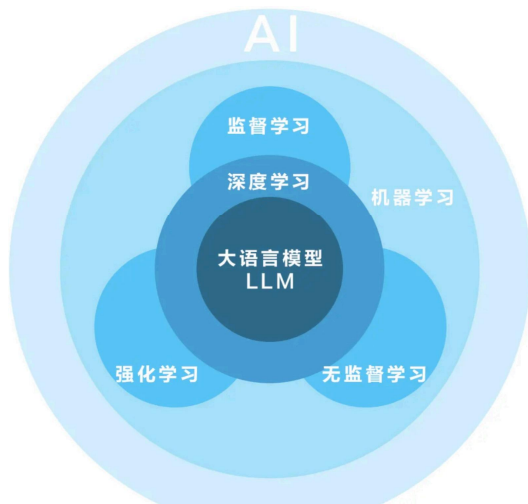
代码能力要求：中低，AI/数学基础要求：中低

1. 传统机器学习的算法思想和核心理念，是深度学习的基石；只有知晓基础概念和原理，才能理解为什么这么做，以及如何才能更好；
2. 新的进展层出不穷，但了解本质与原理后，才能理解为什么以及未来的趋势。盲目追新未必是一件好事；
3. 如果您完全没有技术基础：
 - 关注核心概念，理解设计思路，**了解是什么，以及为什么**
4. 如果您有技术背景，但对机器学习不了解：
 - 尽量理解 核心概念和推导逻辑，**尝试体会为什么这么用，以及效果不佳可能的原因是什么，直到努力调整的方向；**
5. 如果你有机学习基础，可以**查漏补缺，在理解为什么，直到调整的方向后，可以尝试去纠正。**
6. 不要纠结于数学公式，先理解这样带来了什么，很多都是基于数学理论设计的trick。
7. 学习路线：**有大概得了解与认知后，结合工作中的实际场景，利用 AI 帮你解惑、迭代、调优、应用。**
8. **先不求甚解，后温故知新。**

开始上课！

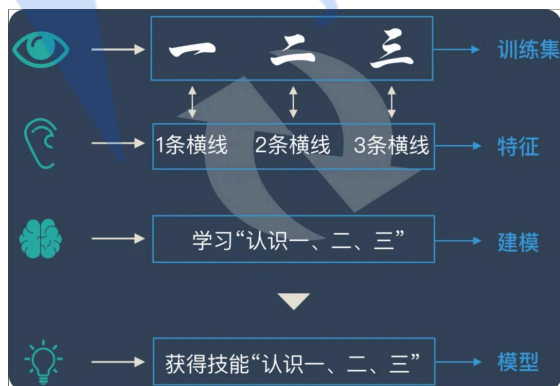
人工智能-机器学习-深度学习之间的关系

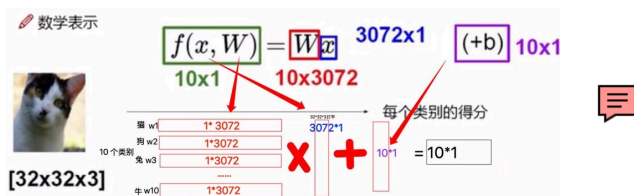
- 人工智能是模拟人类智能的广泛技术领域
- 机器学习 (Machine Learning, ML)：人工智能的一个子领域，专注于让计算机通过数据学习和改进性能，无需显式编程。
- 深度学习 (Deep Learning, DL)：机器学习的一个更具体的子集，基于人工神经网络，特别擅长处理复杂的非线性问题。
- 大语言模型：大语言模型 (LLM) 是一种基于大量数据训练的统计语言模型，可用于生成和翻译文本和其他内容，以及执行其他自然语言处理 (NLP) 任务。
- 生成式 AI：生成式 AI (生成式人工智能) 是指使用 AI 来创作新内容，例如文本、图片、音乐、音频和视频。



- 特征工程：特征工程是机器学习流程中至关重要的环节，其核心目标是通过数据转换和特征构造提升模型性能。
- sklearn：Scikit-learn（简称sklearn）是Python中最流行的开源机器学习库之一，专注于提供简单高效的数据挖掘和数据分析工具。
- Orange3 是一个开源的 Python 数据挖掘和机器学习工具箱，以其可视化编程界面和丰富的功能著称，适合从新手到专家的各类用户。
- 模型评估：机器学习流程中验证模型性能的关键环节，其核心目标是通过量化指标和可视化分析判断模型的泛化能力、稳定性和适用性。

机器学习的流程

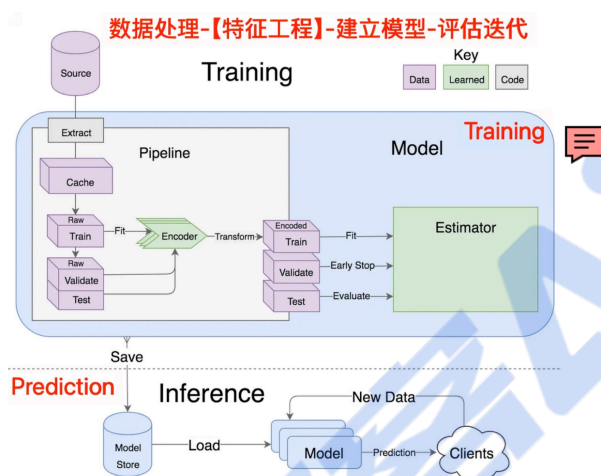




- 问题：
 - 问题 1：机器学习中的学习是指什么？
 - 问题 2：公式中已知的是哪些？未知的是哪些？
 - 问题 3：参数的数据量怎么算？

机器学习的步骤

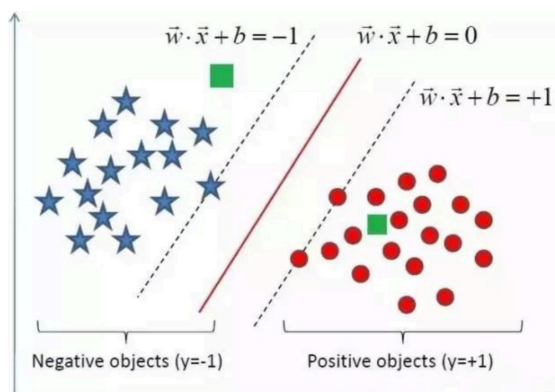
五大步骤：数据处理--> 特征工程 --> 建立模型 --> 评估迭代 --> 上线应用--> 持续优化



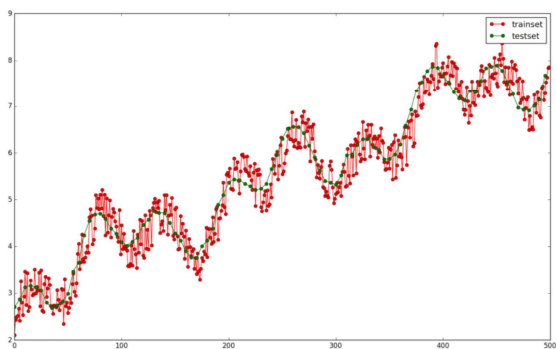
机器学习中的任务

机器学习的任务类别

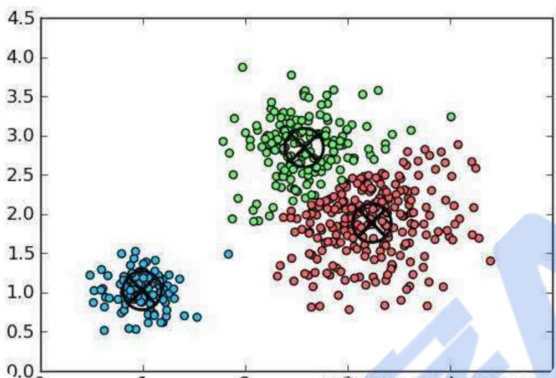
- 分类 (classification)：分类需要先找到数据样本点中的分界线，再根据分界线对新数据进行分类，分类数据是离散的值，比如图片识别、情感分析等领域会经常用到分类任务。比如把人分为好人和坏人之类的学习任务；分析图片是具体是哪种动物等等；
 - 二分类 (binary classification)：只涉及两个类别的分类任务。
 - 多分类 (multi-class classification)：涉及多个类别的分类。



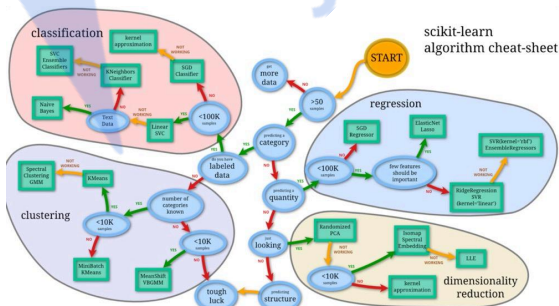
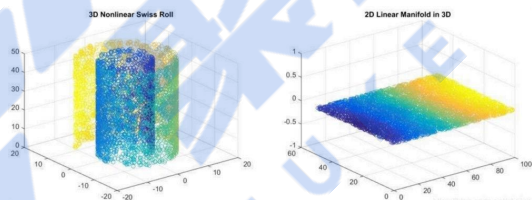
- 回归 (regression) : 回归是对已有的数据样本点进行拟合, 再根据拟合出来的函数, 对未来进行预测。回归数据是连续的值预测值是连续值, 比如你的好人程度达到了0.9; 房价预测、股价预测等等;



- 聚类: 聚类是根据样本之间的相似度, 将一批数据划分为N个组。比如用户分组、异常值检测等领域会用到聚类任务。

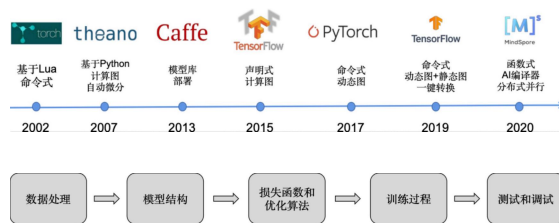


- 降维: 减少数据的维度, 对数据进行降噪、去冗余, 方便计算和训练; 如数据预处理, 减少一些对模型准确率影响很小维度, 可以提高计算效率。再如图表可视化, 我们进行数据分析时, 通常会将高维模型降为三维或二维图表, 便于直观分析。



机器学习系统编程模型的演进

随着机器学习系统的诞生, 如何设计易用且高性能的API接口就一直成为了系统设计者首要解决的问题。



机器学习系统编程模型的首要设计目标是：对开发者的整个工作流进行完整的编程支持。一个常见的机器学习任务一般包含数据处理、模型定义、优化器定义、训练、测试和调试等几大阶段。

- 数据处理：首先，用户需要数据处理API来支持将数据集从磁盘读入。
- 模型定义：用户需要模型定义API来定义机器学习模型。这些模型带有模型参数，可以对给定的数据进行推理。
- 优化器定义：模型输出需要和用户的标记进行对比，这个对比差异一般通过损失函数（Loss function）来进行评估。并根据损失来引入（Import）和定义各种优化算法（Optimisation algorithms）来计算梯度（Gradient），完成对模型参数的更新。
- 训练：给定一个数据集，模型，损失函数和优化器，用户需要训练API来定义一个循环（Loop）从而将数据集中的数据按照小批量（mini-batch）的方式读取出来，反复计算梯度来更新模型。
- 测试和调试：训练过程中，用户需要测试API来对当前模型的精度进行评估。当精度达到目标后，训练结束。

PyTorch 是一种开源深度学习框架，以出色的灵活性和易用性著称。这在一定程度上是因为与机器学习开发者和数据科学家所青睐的热门 Python 高级编程语言兼容。

PyTorch

PyTorch是一个由Facebook开源的深度学习框架，是目前市场上最流行的深度学习框架之一。PyTorch的应用范围非常广泛，包括图像和语音识别、自然语言处理、计算机视觉、推荐系统等领域。PyTorch具有易于使用、灵活性高和代码可读性好等特点，使得它成为深度学习研究和应用的首选框架之一。PyTorch使用 Python 编写，因此对于大多数机器学习开发者而言，学习和使用起来相对简单。PyTorch 的独特之处在于，它完全支持 GPU，并且使用反向模式自动微分技术，因此可以动态修改计算图形。这使其成为快速实验和原型设计的常用选择。

环境准备

我们需要配置一个环境来运行 Python、Jupyter Notebook、相关库以及运行本书所需的代码，以快速入门并获得动手学习经验。

conda安装

Miniconda 是一个轻量级的 Anaconda 发行版，专为需要灵活管理 Python 环境和包的用户设计。以下是关于 Miniconda 的详细信息：Miniconda 的特点

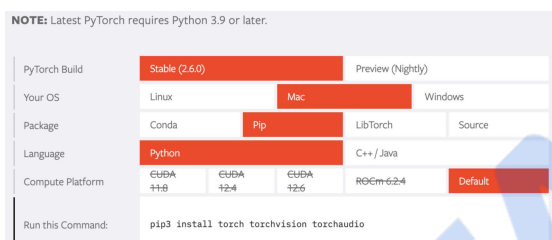
- 轻量级：仅包含 Conda 包管理器和 Python 的最小安装包，占用空间小。
- 灵活性：允许用户根据需要安装额外的库，避免不必要的依赖。
- 跨平台：支持 Windows、macOS 和 Linux 系统。

- 安装 conda, Python工具大全, 方便管理多个 Python 环境, 必须选择跟自己环境配套的版本。
- <https://www.anaconda.com>
- 网速慢的, 可以参考国内源, 也可以去这里看看:
- <https://pypi.org/project/torch/>
- <https://mirrors.tuna.tsinghua.edu.cn/anaconda/miniconda/>

	Anaconda Distribution	Miniconda
Created and published by Anaconda	Yes	Yes
Has conda	Yes	Yes
Has Anaconda Navigator	Yes	No
# of packages	300+	< 70
Install space required	~4.4 GB	~480 MB

Pytorch 安装

PyTorch的安装, 结合自己想要安装的环境, 动态选择即可。 <https://pytorch.org/>



#环境初始化

```
~/miniconda3/bin/conda init
```

#创建虚拟环境

```
conda create --name deeplearning python=3.9
```

#激活环境

```
conda activate deeplearning
```

#安装插件

```
pip install torch==1.12.0 torchvision==0.13.0 numpy==1.21.5  
matplotlib==3.5.1 requests==2.25.1 pandas==1.2.4  
python -m pip install -U pip  
python -m pip install -U matplotlib
```

#退出环境

```
conda deactivate
```

#列出当前有哪些环境

```
conda env list
```

#删除环境

```
conda env remove deeplearning
```

数学基础

标量

定义: 仅用单一数值即可完全描述的量, 没有方向, 只有大小 (或“量级”)。

数学表示：标量是零阶张量（0-dimensional），用单个实数或复数表示。例子：温度（如 25°C）、质量（如 5 kg）、时间（如 10 秒）

向量

- 定义：既有大小又有方向的量，可看作一维有序数组，是一阶张量。
- 特点：
 - 可进行加减、标量乘法、点积、叉积等运算。
 - 方向性体现在其分量随坐标系变化（如速度、力）。
 - 机器学习中的特征向量（如用户画像的数值表示）

张量 (Tensor)

定义：广义的多维数组，可表示任意阶数的数据。标量是 0 阶张量，向量是 1 阶张量，矩阵是 2 阶张量，更高维的数组为高阶张量。

- 数学表示：
 - 阶数（秩）由索引数量决定。例如，3 阶张量 T 可表示为 T_{ijk} 。
- 例子：
 - 矩阵（2 阶张量）：图像灰度像素矩阵 28×28 。
 - 3 阶张量：RGB 彩色图像（高度 $\times$ 宽度 $\times$ 3 颜色通道）。
 - 4 阶张量：视频数据（时间 $\times$ 高度 $\times$ 宽度 $\times$ 颜色通道）。

小结

三者的核心区别

属性	数量 (标量)	向量	张量
阶数 (维度)	0 阶 (零维)	1 阶 (一维)	任意阶 (n-维)
数据结构	单个数值	一维数组	多维数组
方向性	无方向	有方向	可能包含多个方向
数学运算	普通算术 (+-x*)	向量加减、点积、叉积	张量积、收缩、卷积等
物理意义	纯量 (如能量)	方向性量 (如力)	复杂关系 (如应力)

- 标量是单一数值，向量是带方向的数列，张量是广义的多维数据结构。
- 张量是标量和向量的高阶推广，能表示更复杂的物理量或数据关系。
- 在深度学习，张量是核心数据结构，例如图像（3 阶张量）和批量数据（4 阶张量）。
- PyTorch 和 TensorFlow中都是用 Tensor 的概念，MXNet 中使用 ndarray 的概念。

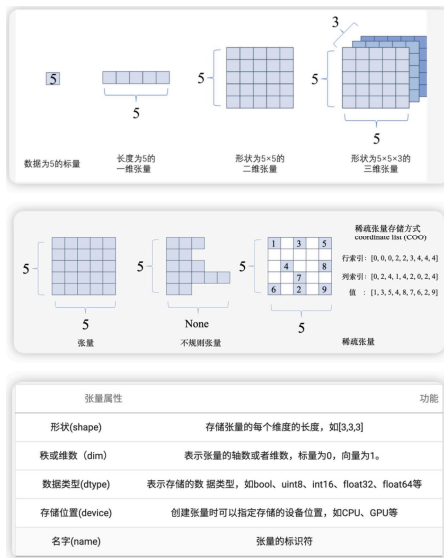
PyTorch中的张量

首先，我们导入torch。请注意，虽然它被称为PyTorch，但是代码中使用torch而不是pytorch。

PyTorch 中张量Tensors定义

- 张量表示一个由数值组成的数组，这个数组可能有多个维度。
- 具有一个轴的张量对应数学上的向量（vector）；
- 具有两个轴的张量对应数学上的矩阵（matrix）；
- 具有两个轴以上的张量没有特殊的数学名称。

在 PyTorch 中，标量、向量、张量与数学界的标识基本一致。为了存储方便，在计算机上存储时，通常有不同的存储表达。



代码实际操作

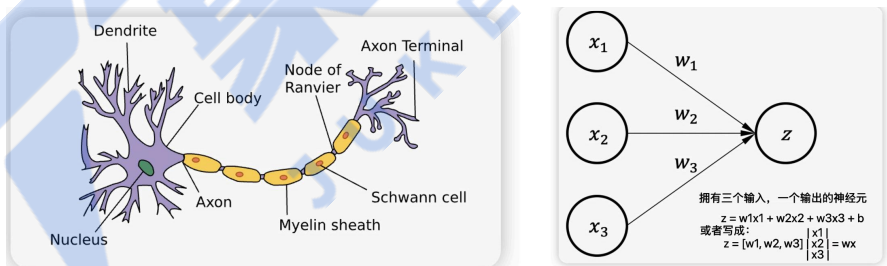
- 01-张量的基本操作.ipynb
- 02-数据预处理.ipynb
- 03-线性代数.ipynb

机器学习中的关键术语及其含义

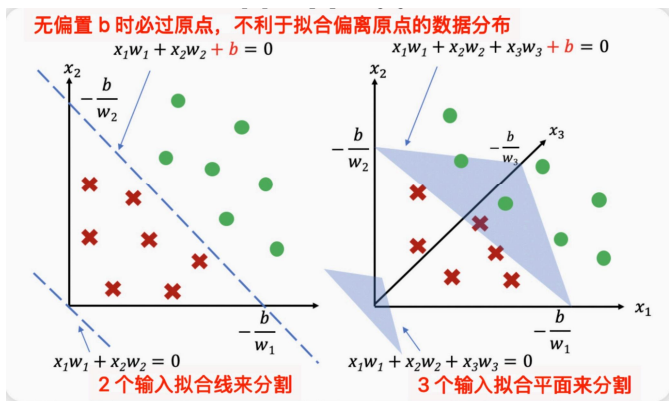
神经元及神经网络

机器学习中的神经网络是一种模仿生物神经网络的结构和功能的数学模型或计算模型。它是指按照一定的规则将多个神经元连接起来的网络。

神经网络是一种运算模型，由大量的节点（或称神经元）之间相互联接构成。每个节点代表一种特定的输出函数，称为激励函数。

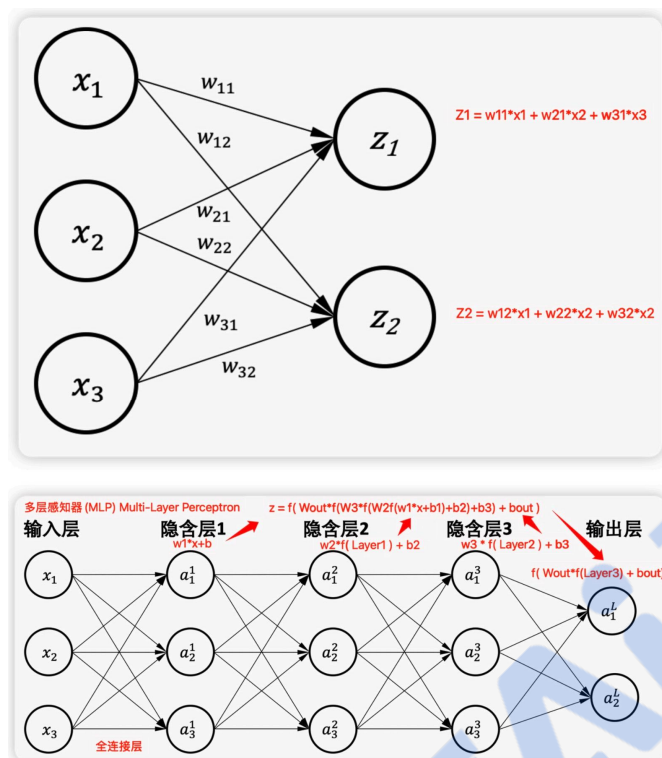


偏置项的作用



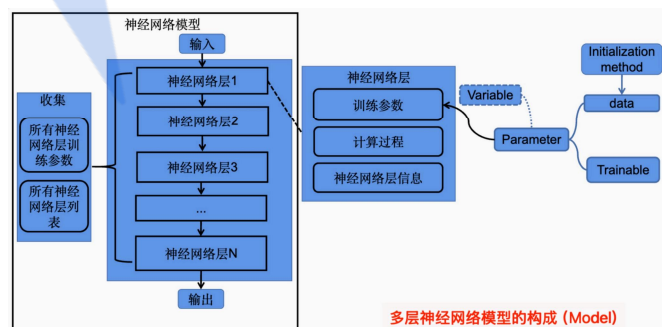
多层感知机

多个神经元可以组合一起，形成多层感知机。多层感知器（Multi-Layer Perceptron, MLP）：通过叠加多层全连接层来提升网络的表达能力。相比单层网络，多层感知器有很多中间层的输出并不暴露给最终输出，这些层被称为隐含层（Hidden Layers）。



下图描述了神经网络构建过程中的基本细节。

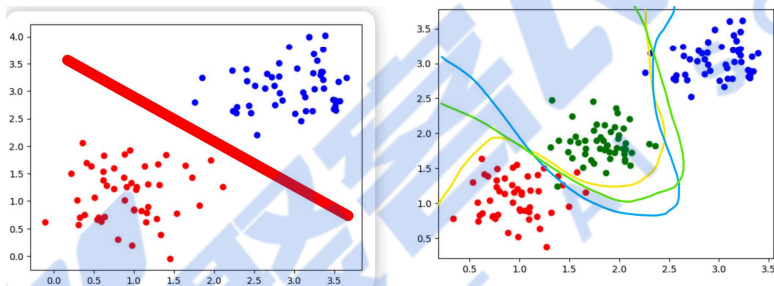
- 基类需要初始化训练参数、管理参数状态以及定义计算过程；神经网络模型需要实现对神经网络层和神经网络层参数管理的功能。
- 在机器学习编程库中，承担此功能有MindSpore的Cell、PyTorch的Module。Cell和Module是模型抽象方法也是所有网络的基类。现有模型抽象方案有两种，
 - 一种是抽象出两个方法分别为Layer（负责单个神经网络层的参数构建和前向计算），Model（负责对神经网络层进行连接组合和神经网络层参数管理）；
 - 另一种是将Layer和Model抽象成一个方法，该方法既能表示单层神经网络层也能表示包含多个神经网络层堆叠的模型，Cell和Module就是这样实现的。



样本、目标函数、损失函数、特征及训练

- 样本 (Sample) 定义: 样本是数据集中的单个实例或数据点, 通常由一组特征 (自变量) 和一个标签 (因变量) 组成。举例: 在房价预测中, 一个样本可能包含房屋的面积、卧室数量、位置等特征, 以及对应的房价标签。
- 标签 (Label) 定义: 标签是与样本关联的目标值或类别, 用于监督学习中指导模型学习。举例: 在垃圾邮件分类中, 标签可以是“垃圾邮件”或“非垃圾邮件”。 代表Y
- 自变量 (Independent Variable) 定义: 自变量是用于预测或解释目标变量的输入特征, 也称为特征或预测变量。举例: 在预测学生成绩时, 自变量可以是学习时间、家庭收入等。 代表W和B
- 目标函数 (Target Function) 定义: 目标函数是模型训练过程中优化的目标, 通常由损失函数构成, 用于衡量模型性能。举例: 在线性回归中, 目标函数是最小化均方误差 (MSE)。 LOSS
- 损失函数 (Loss Function) 定义: 损失函数用于量化模型预测值与真实值之间的差异, 是目标函数的核心组成部分。用来衡量单个样本中计算值与标签值的差异。举例: 在分类问题中, 常用的损失函数是交叉熵损失。
- 代价函数 (Cost Function) 定义: 代价函数是损失函数在所有样本上的平均值, 用于衡量模型在整个数据集上的性能。损失函数与代价函数的区别在于, 损失函数只适用于单个训练样本, 而代价函数是参数的总代价。举例: 在逻辑回归中, 代价函数是交叉熵损失的平均值。
- 特征 (Feature) 定义: 特征是描述样本的属性或变量, 用于模型的输入。举例: 在图像分类中, 特征可以是像素值或提取的边缘信息。
- 模型 (Model) 定义: 模型是通过机器学习算法从数据中学习到的数学表示, 用于对新数据进行预测。举例: 决策树模型可以根据输入特征决定输出类别。
- 训练数据 (Training Data) 定义: 训练数据是用于训练机器学习模型的数据集, 通常包含输入特征和对应的标签。举例: 在预测房价的模型中, 训练数据可能包括房屋特征及其相应的价格。
- 测试数据 (Testing Data) 定义: 测试数据是用于评估模型在未知数据上表现的数据集。举例: 在训练垃圾邮件过滤器后, 可以在以前从未见过的电子邮件上对其进行测试。
- 正则化 (Regularization) 定义: 正则化是一种技术, 用于防止模型过度拟合, 通过在损失函数中添加惩罚项来限制模型的复杂度。举例: L2正则化通过在损失函数中添加权重平方和来限制权重的大小。
- 学习率 (Learning Rate) 定义: 学习率是一个超参数, 控制模型权重相对于损失梯度的更新程度。举例: 在神经网络中, 学习率决定了模型在训练期间从错误中学习的速度。 目前默认的优化器Adamw, 它可以根据损失自动调整学习率
- Epoch 定义: 一个epoch是指在模型训练过程中对整个训练数据集进行一次完整的遍历。举例: 如果有1000个训练样本, 1个epoch意味着模型已经看过所有1000个样本一次。 经验: 设置的特别大, 通过观察损失或者通过验证数据, 来评估是否达到要求, 之后手动停止

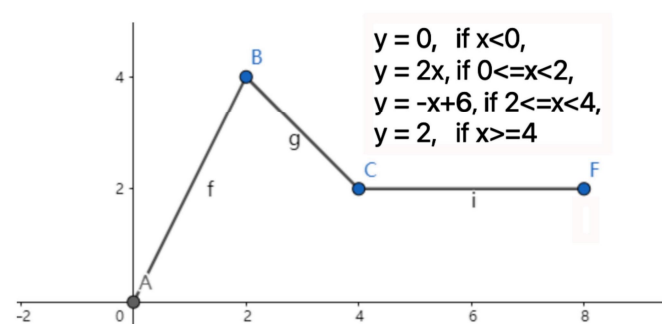
- 超参数 (Hyperparameter) 定义：超参数是在训练之前设置的参数，用于控制学习过程和模型结构。举例：学习率、批量大小 (batch size)、神经网络的层数和每层的神经元数量等都是常见的超参数。
- 回归问题：回归问题是机器学习中的一种任务，其目标是预测一个连续值作为输出。
 - 目标变量：回归问题中的目标变量是连续的，可以取任何实数值。
 - 特征：用于预测目标变量的输入变量，可以是连续的或离散的。
- 线性回归问题：线性回归是一种回归模型，假设输入变量 (特征) 和输出变量 (目标) 之间存在线性关系。它通过找到一条最佳拟合直线来模拟这种关系。
- 逻辑回归问题：逻辑回归是一种用于分类问题的统计方法，尽管名字中包含“回归”，但它实际上是一种分类算法。
 - 定义：它通过逻辑函数 (Sigmoid函数) 将输入特征映射到0到1之间的概率值，表示属于某个类别的可能性。
- 训练：找到一组参数值Weight(面积)、Weight(age)、b，能够使得在给定的训练数据集上，所产生的集体误差最小。
- 预测：当用户给定房屋面积、年龄后，能够给出对应的房价。这个房价越贴近于真实值，表示模型效果越好。
- 数据分布：训练的过程就是找到一个函数，能够匹配数据的分布的过程。



非线性激活函数

激活函数：人工神经网络中的一个关键组件，它负责将神经元的输入信号转换为输出信号，从而引入非线性特性。如果没有激活函数，神经网络的每一层都只是线性变换的叠加，整个网络将退化为一个简单的线性模型，无法拟合复杂的数据分布或解决非线性问题。

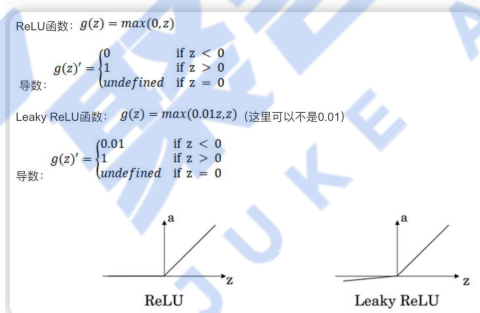
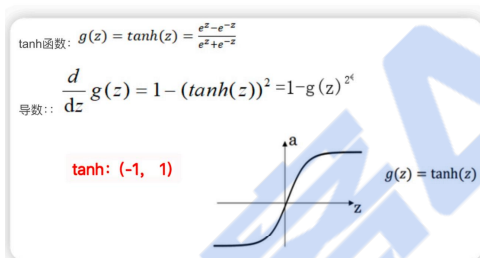
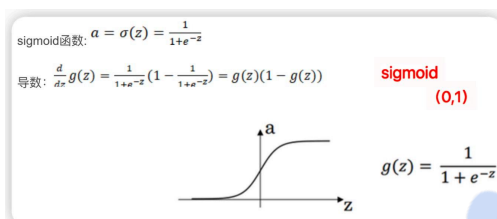
讨论1：为什么激活函数必须要有非线性的？



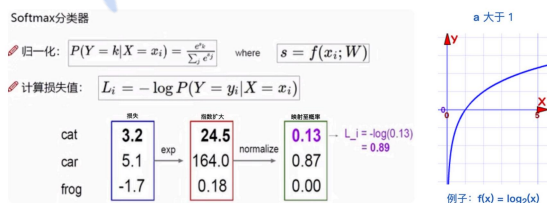
- 激活函数对模型至关重要：引入非线性、增强表达能力、控制输出范围、促进梯度传播、模拟生物神经元的激活机制。

常见的激活函数

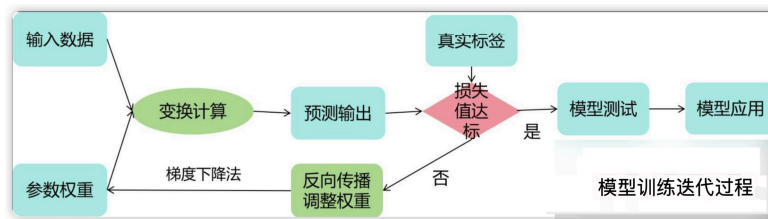
- 常见激活函数：
 - Sigmoid函数：将输入映射到(0,1)之间，常用于二分类问题，但容易导致梯度消失问题。
 - Tanh函数：将输入映射到(-1,1)之间，解决了Sigmoid的零均值问题，但仍存在梯度消失问题。
 - ReLU函数：在输入为正时输出输入值，输入为负时输出0，计算简单且能有效缓解梯度消失问题，但可能导致“神经元死亡”问题。



- Softmax函数：将输入映射为概率分布，常用于多分类问题的输出层。

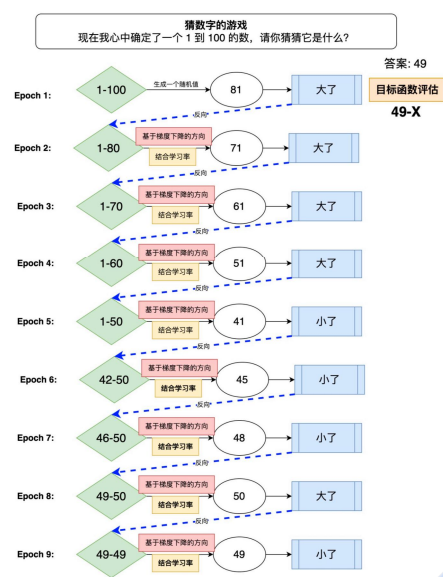


典型的模型训练过程



反向传播机制

用一个实际的例子来理解机器学习和反向传播



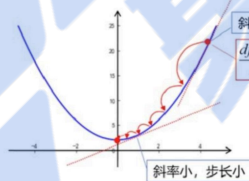
学习率的影响

梯度下降法

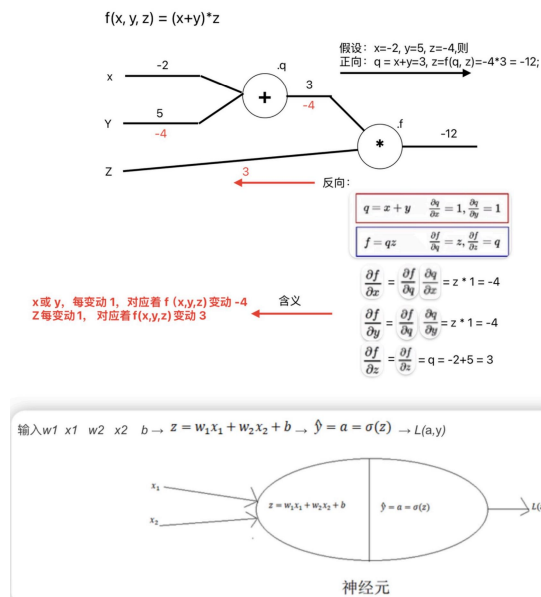
步长太小，迭代次数多，收敛慢
步长太大，引起震荡，可能无法收敛

$$\text{步长} = \eta \frac{df(x)}{dx} \quad \eta: \text{学习率}$$

$$x^{(k+1)} = x^{(k)} - \eta \frac{df(x)}{dx}$$



正向传播与反向传播的计算



由单个样本至多个样本的反向传播的计算

- 1) (单个样本) 正向传播的计算过程: $\hat{y}$ 是根据输入参数计算得到的结果, 而 y 是样本数据自带的实际结果, 根据下面的公式即可得到代价。
- 2) (单个样本) 反向传播的计算过程:

1 正向传播

- ① $z = w_1x_1 + w_2x_2 + b$
- ② $\hat{y} = a = \sigma(z)$, $\sigma(z) = \frac{1}{1+e^{-z}}$
- ③ $L(a, y) = -y\log(a) + (1-y)\log(1-a)$

2 反向传播

- ① 首先反向计算出代价函数关于 a 的导数 $\frac{dL(a, y)}{da} = -y/a + (1-y)/(1-a)$
- ② 然后反向计算 dz , $dz = \frac{dL(a, y)}{dz} = \frac{dL}{da} \cdot \left(\frac{da}{dz}\right) = \left(-\frac{y}{a} + \frac{(1-y)}{(1-a)}\right) \cdot a(1-a) = a - y$
- ③ 反向计算 dw_1 、 dw_2 和 db , dw_1 表示 $\frac{\partial L}{\partial w_1} = x_1 \cdot dz$, dw_2 表示 $\frac{\partial L}{\partial w_2} = x_2 \cdot dz$, $db = dz$

- 3) 多个样本的正向传播和反向传播

1 正向传播

- ① $Z = W^T X + b$ (X 的维度 (n_s, m) , Z 的维度 $(1, m)$, W 的维度 $(n_s, 1)$)
- ② $\hat{Y} = A = \sigma(Z)$
- ③ $J(w, b) = \frac{1}{m} \sum_{i=1}^m L(a^{(i)}, y^{(i)})$

2 反向传播: 从单个样本 推理到多个样本, 也就是矩阵化的过程

- ① $dZ = A - Y$
- ② $dW = (1/m) * X * dZ^T$
- ③ $db = (1/m) * \text{np.sum}(dZ)$
- ④ $W = W - \alpha * dW$ (α 是学习率)
- ⑤ $b = b - \alpha * db$

链式法则